

Q1:

Row Key & Sharding:

My designed Row Key is: hash_prefix#City#Make#VIN

For example: 07#Seattle#TESLA#5YJ3E1EA...

The reason for this design is that the hash_prefix first disperses the data, allowing new data to be distributed across different nodes. This avoids the hotspotting problem caused by many data sets being written to the same place.

If only ordered fields are used as Row Keys, such as sorting only by year or only by VIN, many new data sets might consistently fall into the same area, causing excessive pressure on a particular node.

I believe the advantage of my design is more even write distribution and greater system stability. However, its disadvantage is that if I want to query by City or Make, the query is not as straightforward because data from the same city or brand might be distributed across different partitions, so the system might need to query multiple places to find all the results

Q2:

I will divide this dataset into four column families.

The first family is `vehicle\_info`, including VIN (1-10), Model Year, Make, Model, and Electric Vehicle Type. This family mainly contains basic vehicle information.

The second family is `location\_info`, including County, City, State, Postal Code, Vehicle Location, and 2020 Census Tract. This family mainly contains location-related information.

The third family is `policy\_info`, including Clean Alternative Fuel Vehicle (CAFV) Eligibility, Legislative District, and DOL Vehicle ID. This family mainly contains policy and registration-related information.

The fourth family is `utility\_info`, including Electric Range and Electric Utility. This family mainly contains range and electricity service-related information.

The advantage of this grouping is that similar information is grouped together, making data retrieval more convenient for the system. For example, if only basic vehicle information is needed, only `vehicle\_info` needs to be read, without needing to retrieve location and electricity company information. This reduces unnecessary data retrieval, improves distributed read performance, and makes the overall data structure clearer.

Q3:

For Hashed Shard Key (e.g., hashing VIN)

Advantages:

- More even data distribution
- Less prone to hotspots
- Write pressure is more evenly distributed across nodes

Disadvantages:

- Query routing is less direct
- Inconvenient for range queries based on certain fields
- A single query may require accessing multiple shards

Range-Based Shard Key (e.g., by Model Year)

Advantages:

- Easier to query by year
- Clearer query routing
- More friendly for range queries

Disadvantages:

- Prone to hotspots
- If a lot of new data is concentrated in the latest year, the corresponding shard will experience high pressure
- Cluster load may not be evenly distributed

In general, if the system prioritizes even write distribution and scalability, I would prefer Hashed Shard Key. If the system prioritizes range analysis by year, then Range-Based Shard Key would be more suitable.

Part4:

I used a Python script to test the response time of ``/insert-fast`` 50 times and ``/insert-safe`` 50 times respectively.

The test results are as follows:

- `insert-fast` average latency: 5.62 ms
- `insert-safe` average latency: 24.95 ms

The results show that ``/insert-fast`` is significantly faster, and `insert-safe` is slower.

The reason is that `/insert-fast` only needs the primary node to confirm the write before returning success, so the waiting time is relatively short. While `/insert-safe` requires a majority of replica nodes to confirm the write before returning, so it waits for more replication processes, resulting in higher latency. This illustrates that in distributed systems, there is often a trade-off between speed and reliability.

Real-world Scenario

In real-world scenarios, I would proactively choose the faster primary-only write for click logs, ad impression records, and user browsing behavior statistics. This is because this type of data is typically large in volume, and low latency and high throughput are paramount.

Even if the master node suddenly crashes, causing the loss of a small amount of the latest data, the overall system usually still functions normally, because this data is primarily used for statistical analysis, not critical data like bank transfers or payment orders.

Therefore, in these scenarios, I would prefer to use the faster `/insert-fast`, because speed is more important than "every single piece of data must be absolutely not lost."